

MACHINE INTELLIGENCE (2025)

Prof. Andres Perez-Uribe

Practical work prepared Hector-Fabio Satizabal & Andres Perez-Uribe

REINFORCEMENT LEARNING

This practical work has the aim of learning the basic concepts of reinforcement learning. We will focus on the tabular version of these algorithms and will apply use them to let an agent learn to behave in a simple world, guided by rewards and punishments. We will use a library called **gym** developed by OpenAI.

INSTRUCTIONS

FROZEN LAKE PROBLEM

Run the notebook implementing the Q-learning algorithm of reinforcement learning to let an agent learn by trial and error how to go from a starting position to a target position in a grid world, while avoiding to fall into holes. The environment is a simulation of a frozen lake, and the holes are formed by melting ice. Analyze the code, find where the learning is taking place, the hyper-parameters of the learning algorithms, etc

1. Identify and copy the python code that performs the value function adaptation (i.e., the modification of Q-values). Is it the same as the one presented in the lesson ? does it correspond to SARSA or Q-learning ? what is the difference between those learning algorithms ? Explain.
2. Using a 4x4 environment modify the code to stop the learning process every 100 epochs to evaluate the performance of the agent. To do this, you can run 100 episodes (Each time the agent is placed in a starting position and tries to reach the target. Take note of how many times it reaches the target) while putting alpha to zero (no learning) and epsilon to zero (no exploration). Generate a plot of the evolution of the performance as a function of iterations.
3. Modify gamma (the discount factor) to 0.9, and alpha (learning rate) to 0.05 and 0.01. Compare the resulting plots of performance vs. epochs of the 6 combinations of hyper-parameters and provide your observations.
4. Run a 5x5 environment. Track the learning process to determine the minimum number of epochs needed to allow the agent to have a good performance. Can it reach a perfect performance ? Is it more rapid than the exhaustive search? Explain.
5. Try to modify the reward function by providing the agent with a punishment of $r=-0.1$ for each step it takes to try to reach the target instead of giving it a reward $r=+1$ when it reaches the target. Help: <https://medium.com/@ym1942/create-a-gymnasium-custom-environment-part-1-04ccc280eea9>

TAXI PROBLEM

Run the notebook and analyze the code.

1. Using the taxi environment modify the code to stop the learning process every 100 epochs to evaluate the performance of the agent. To do this, you can run 100 episodes (Each time the agent is placed in a starting position and tries to reach the target. Take note of how many times it reaches the target) while putting alpha to zero (no learning) and epsilon to zero (no exploration). Generate a plot of the evolution of the performance as a function of iterations.
2. Perform hyper-parameter tuning considering the value of epsilon (exploration), the discount factor and the number of epochs. Present your results.
3. Find how the state of the agent is computed from the observed variables. For a trained agent list the states where the agent performs the “dropoff” and “pickup” actions and verify if the behavior is the right one. You may use the `env.decode()` function. Explain.
4. For a trained agent, use the learned strategy to pickup one by one, three passengers at locations 1, 2 and 3 and drop them at location 0. Perform 100 simulations and for each one, compute the percentage of success and the mean steps required to complete the task.